

A3-PS022-TC131

eBPF/XDP Rate Limiting for DNS Query Floods in a Lab

TEAM-DAAKY

AIORI Hackathon 3

October 14, 2026

Problem Description

PS-022: eBPF/XDP Rate Limiting for DNS Query Floods in a Lab.

High-volume UDP DNS query floods saturate Linux servers because the operating system kernel spends substantial CPU resources allocating socket buffers before packets reach user space. eXpress Data Path (XDP) runs eBPF bytecode directly inside the network interface driver to process or drop packets before kernel memory allocation. In this project, you will build an XDP filter on a Linux host pair that performs bounded UDP DNS parsing. By applying per-source and global rate limits across three synthetic query flood patterns, you will benchmark CPU utilization, packet drops, and legitimate query success against both nftables and no-filter baselines.

Background

- DNS translates domain names into IP addresses and supports essential network services.
- DNS commonly uses UDP, making high-volume query traffic a potential resource-exhaustion concern.
- Conventional packet processing can incur overhead before traffic reaches the application.
- XDP allows eBPF programs to process packets at an early point in the Linux networking path.
- Rate limiting can restrict excessive requests while allowing traffic within configured limits.

Literature & RFC Review

1. Linux kernel eBPF documentation

- Technical foundation for eBPF programs and their interaction with the Linux kernel.
- Reference: docs.kernel.org/bpf

2. XDP project tutorial

- Practical reference for learning and implementing XDP packet-processing programs.
- Reference: xdp-project/xdp-tutorial

3. nftables rate limiting and meters

- Reference for conventional firewall rate-limiting policies.
- References: Meters and Rate-limiting examples

4. DNS performance testing

- DNS-OARC dnsperf provides a reference tool for generating DNS performance workloads.
- Reference: DNS-OARC/dnsperf

Proposed Solution

Core components

1. **XDP packet filter:** Inspect incoming packets before normal Linux network-stack processing.
2. **Bounded packet parser:** Validate packet boundaries and identify supported IPv4/UDP DNS traffic.
3. **Per-source token bucket:** Limit requests from an individual source.
4. **Global token bucket:** Enforce a host-wide DNS request budget.
5. **Dynamic blacklist:** Temporarily block sources after repeated rate-limit violations.
6. **Hybrid firewall:** Allow passed traffic to continue through the Linux stack, where applicable nftables rules can provide additional protection.
7. **Monitoring:** Record allowed packets, drops, limit violations, and blacklist activity.

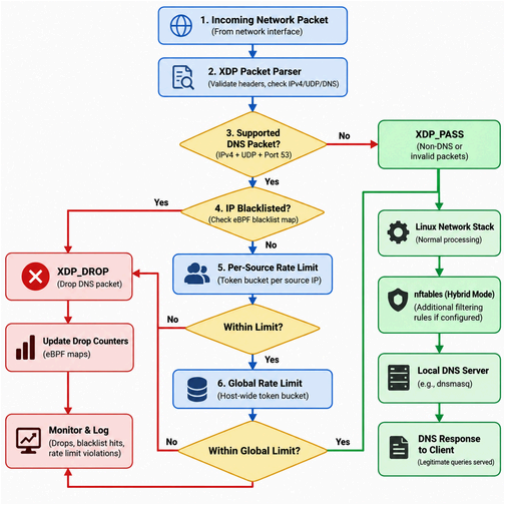
Packet decisions: XDP_PASS for allowed or out-of-scope traffic; XDP_DROP for traffic rejected by the implemented policy.

Optimization Proposed

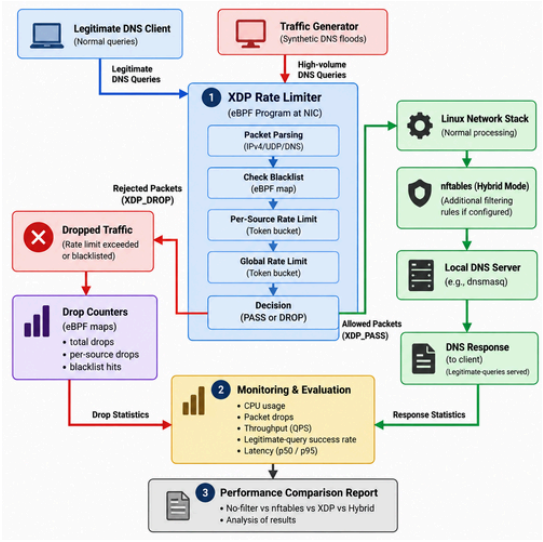
- **Early Packet Filtering:** Use eBPF/XDP to drop excessive DNS traffic before it enters the normal Linux networking stack.
- **Rate Limiting:** Apply per-source and global token buckets to control excessive DNS queries.
- **Dynamic Blacklisting:** Temporarily block IP addresses that repeatedly exceed rate limits.
- **Hybrid Firewall:** Combine XDP with nftables for additional traffic filtering.
- Performance Monitoring: Compare CPU usage, packet drops, throughput, and legitimate-query success.

Expected Outcome: Efficient DNS flood handling while maintaining service for legitimate users. These improvements will be validated through controlled experiments.

Solution Architecture & Design



Data Flow Diagram



Deployment Timeline

- **Phase 1: Lab Setup — Verify Linux VMs, network isolation, and DNS connectivity.**
- **Phase 2: XDP Setup — Attach a pass-only XDP program and verify safe operation.**
- **Phase 3: Rate Limiting — Implement bounded packet parsing, per-source and global token buckets.**
- **Phase 4: Enhancements — Add dynamic IP blacklisting and optional nftables integration.**
- **Phase 5: Testing — Run three synthetic DNS traffic scenarios in the controlled lab.**
- **Phase 6: Evaluation — Measure CPU usage, packet drops, throughput, and legitimate-query success.**
- **Phase 7: Final Demonstration — Compare results, document limitations, and present findings.**

Bibliography I

[1] A Linux Kernel Documentation. “BPF Documentation.” In: Linux Kernel Documentation. Available at: *https://docs.kernel.org/bpf/*.

[2] XDP Project. “XDP Tutorial.” In: GitHub Repository. Available at: *https://github.com/xdp-project/xdp-tutorial*.

Thank You for Your Attention!